PHASE4_EXECUTION_PLAN

PHASE 4 EXECUTION PLAN - INTEGRATION & POLISH

Production Readiness & Launch Preparation

🎯 DAY 1: DISABLED MODULE RESOLUTION

Immediate Action Items

1. Restore SSH Worker Functionality

```
# Investigate backup file status
ls -la pkg/sshworker/

# Compare backup with current implementation
diff pkg/sshworker/worker.go pkg/sshworker/worker.go.backup

# Analyze why backup was created
git log --oneline pkg/sshworker/worker.go
git blame pkg/sshworker/worker.go.backup

# Restore functionality if needed
if [ ! -f "pkg/sshworker/worker.go" ] || [ ! -s "pkg/sshworker/
        worker.go" ]; then
    cp pkg/sshworker/worker.go.backup pkg/sshworker/worker.go
    echo "Restored SSH worker from backup"
fi

# Create comprehensive test to prevent regression
cat > pkg/sshworker/worker_regression_test.go << 'EOF'
package sshworker

import (
    "context"
    "testing"
    "time"
```

```
)

func TestWorker_Connectivity(t *testing.T) {
    // Test SSH worker connectivity and basic functionality
    tests := []struct {
        name    string
        config  *WorkerConfig
        wantErr bool
    }{
        {
            name: "valid config",
            config: &WorkerConfig{
                Host:     "localhost",
                Port:     22,
                Username: "test",
                KeyPath:  "/tmp/test_key",
            },
            wantErr: false,
        },
        // More test cases...
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            worker := NewWorker(tt.config)
            err := worker.Connect(context.Background())
            if (err != nil) != tt.wantErr {
                t.Errorf("Worker.Connect() error = %v, wantErr
        %v", err, tt.wantErr)
            }
        })
    }
}
EOF
```

2. Restore Markdown Workflow System

```
# Analyze disabled workflow
ls -la pkg/markdown/workflow.go*

# Compare versions
diff pkg/markdown/workflow.go.bak pkg/markdown/workflow.go 2>/dev/
        null || echo "Current workflow missing"

# Restore if needed
if [ ! -f "pkg/markdown/workflow.go" ] || [ ! -s "pkg/markdown/
        workflow.go" ]; then
    cp pkg/markdown/workflow.go.bak pkg/markdown/workflow.go
    echo "Restored markdown workflow from backup"
fi

# Create comprehensive test
cat > pkg/markdown/workflow_integration_test.go << 'EOF'
```

```
package markdown

import (
    "context"
    "path/filepath"
    "testing"
    "time"
)

func TestWorkflow_EndToEndProcessing(t *testing.T) {
    if testing.Short() {
        t.Skip("Skipping e2e workflow test in short mode")
    }

    workflow := NewWorkflow()
    ctx := context.Background()

    // Test with actual files
    testInput := filepath.Join("testdata", "test_markdown.md")
    testOutput := filepath.Join(t.TempDir(), "output.epub")

    err := workflow.Process(ctx, testInput, testOutput)
    if err != nil {
        t.Fatalf("Workflow.Process() failed: %v", err)
    }

    // Validate output
    if _, err := os.Stat(testOutput); os.IsNotExist(err) {
        t.Error("Expected output file not created")
    }

    // Additional validation for EPUB structure
    validateEPUBStructure(t, testOutput)
}
EOF
```

3. Remove testing.Short() Guards

```
# Find all files with testing.Short() calls
grep -r "testing.Short()" pkg/ --include="*_test.go" | cut -d:
        -f1 | sort -u

# Automated removal and replacement
find pkg/ -name "*_test.go" -exec sed -i.bak '
    /if testing\.Short() {/,/}/ {
        /if testing\.Short()/ {
            s//\/\/ Skipping integration tests temporarily/
        }
        /t\.Skip/ {
            s/t\.Skip/\/\/ t.Skip/
        }
        /}/ {
```

```
            s/}/\/\/ Integration test temporarily disabled/
        }
    }
' {} \;

# Review changes and manually fix as needed
git diff
```

4. Fix Import Dependencies

```
# Run dependency analysis
go mod tidy
go mod verify

# Identify missing dependencies
go build ./pkg/... 2>&1 | grep "no such file" | cut -d'"' -f2 |
        sort -u

# Add missing imports systematically
for pkg in $(go build ./pkg/... 2>&1 | grep "no such file" | cut
        -d'"' -f2 | sort -u); do
    echo "Resolving dependency for: $pkg"
    go get "$pkg"
done

# Re-run tests to verify imports
go test ./pkg/... -short
```

🎯 DAY 2: PERFORMANCE & SECURITY HARDENING

Performance Optimization

1. Memory Leak Detection & Fixes

```
// Create memory profiling test
func TestMemoryUsage_Regression(t *testing.T) {
    // Test for memory leaks in translation workflow

    // Get baseline memory
    var m1 runtime.MemStats
    runtime.GC()
    runtime.ReadMemStats(&m1)

    // Run translation workflow multiple times
    for i := 0; i < 100; i++ {
        translator := NewTranslator()
        err := translator.Translate("test.fb2", "output.epub")
        if err != nil {
```

```
            t.Fatalf("Translation failed: %v", err)
        }
    }

    // Force garbage collection
    runtime.GC()
    time.Sleep(100 * time.Millisecond)
    runtime.GC()

    // Check memory usage
    var m2 runtime.MemStats
    runtime.ReadMemStats(&m2)

    memIncrease := m2.Alloc - m1.Alloc
    if memIncrease > 10*1024*1024 { // 10MB threshold
        t.Errorf("Potential memory leak detected: %d bytes
        increase", memIncrease)
    }
}

// Add to pkg/translator/memory_test.go
```

2. CPU Optimization & Threading

```
// Optimize concurrent processing
func TestConcurrentProcessing_Performance(t *testing.T) {
    // Test optimal worker thread count
    workerCounts := []int{1, 2, 4, 8, 16}

    for _, workers := range workerCounts {
        start := time.Now()

        err := runConcurrentTranslation(workers, 1000)
        if err != nil {
            t.Errorf("Concurrent processing failed with %d
        workers: %v", workers, err)
            continue
        }

        duration := time.Since(start)
        t.Logf("Workers: %d, Duration: %v", workers, duration)

        // Validate performance scales appropriately
        if workers > 1 && duration > time.Duration(1000/
        workers)*time.Millisecond*100 {
            t.Errorf("Performance scaling issue with %d workers",
        workers)
        }
    }
}
```

3. Performance Benchmarking Suite

```go
// pkg/benchmarks/benchmark_suite.go
package benchmarks

import (
    "testing"
    "time"
)

func BenchmarkTranslationProviders_Comparison(b *testing.B) {
    providers := map[string]Translator{
        "openai":    NewOpenAITranslator(),
        "zhipu":     NewZhipuTranslator(),
        "deepseek":  NewDeepSeekTranslator(),
        "anthropic": NewAnthropicTranslator(),
    }

    testText := "Hello world, this is a benchmark test for
        translation performance comparison across different
        providers."

    for name, provider := range providers {
        b.Run(name, func(b *testing.B) {
            b.ResetTimer()
            for i := 0; i < b.N; i++ {
                _, err :=
        provider.Translate(context.Background(), testText,
        "Translate to Serbian")
                if err != nil {
                    b.Fatalf("Translation failed: %v", err)
                }
            }
        })
    }
}

func BenchmarkEbookParsing_FormatComparison(b *testing.B) {
    parsers := map[string]Parser{
        "fb2":  NewFB2Parser(),
        "epub": NewEPUBParser(),
        "pdf":  NewPDFParser(),
        "docx": NewDOCXParser(),
    }

    testFiles := map[string]string{
        "fb2":  "testdata/sample.fb2",
        "epub": "testdata/sample.epub",
        "pdf":  "testdata/sample.pdf",
        "docx": "testdata/sample.docx",
    }

    for format, parser := range parsers {
```

```
        b.Run(format, func(b *testing.B) {
            content, err := os.ReadFile(testFiles[format])
            if err != nil {
                b.Fatalf("Failed to read test file: %v", err)
            }

            b.ResetTimer()
            for i := 0; i < b.N; i++ {
                _, err := parser.Parse(content)
                if err != nil {
                    b.Fatalf("Parsing failed: %v", err)
                }
            }
        })
    }
}
```

Security Hardening

1. Security Audit Implementation

```
// pkg/security/security_audit_test.go
package security

import (
    "testing"
    "strings"
    "unicode"
)

func TestSQLInjection_Prevention(t *testing.T) {
    // Test SQL injection prevention in database operations
    maliciousInputs := []string{
        "'; DROP TABLE users; --",
        "' OR '1'='1",
        "'; INSERT INTO translations VALUES ('hacked'); --",
        "UNION SELECT * FROM sensitive_data --",
    }

    for _, input := range maliciousInputs {
        t.Run("Input: "+input, func(t *testing.T) {
            sanitized := sanitizeInput(input)

            // Check for dangerous SQL patterns
            dangerousPatterns := []string{"'", "--", ";",
        "UNION", "DROP", "INSERT"}
            for _, pattern := range dangerousPatterns {
                if strings.Contains(strings.ToUpper(sanitized),
        pattern) {
                    t.Errorf("SQL injection prevention failed for
        input: %s", input)
                }
```

```
            }
        })
    }
}

func TestXSS_Prevention(t *testing.T) {
    // Test XSS prevention in web interface
    xssPayloads := []string{
        "<script>alert('xss')</script>",
        "javascript:alert('xss')",
        "<img src=x onerror=alert('xss')>",
        "<svg onload=alert('xss')>",
    }

    for _, payload := range xssPayloads {
        t.Run("Payload: "+payload, func(t *testing.T) {
            sanitized := escapeHTML(payload)

            // Check for HTML tag removal
            if strings.Contains(sanitized, "<script>") ||
               strings.Contains(sanitized, "javascript:") ||
               strings.Contains(sanitized, "onerror=") ||
               strings.Contains(sanitized, "onload=") {
                t.Errorf("XSS prevention failed for payload: %s",
        payload)
            }
        })
    }
}

func TestAPIKey_Security(t *testing.T) {
    // Test API key security and validation
    tests := []struct {
        name    string
        key     string
        wantErr bool
    }{
        {"empty key", "", true},
        {"null key", "null", true},
        {"undefined", "undefined", true},
        {"valid key", "sk-1234567890abcdef", false},
        {"key with injection", "sk-'; DROP TABLE api_keys; --",
        true},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            err := validateAPIKey(tt.key)
            if (err != nil) != tt.wantErr {
                t.Errorf("validateAPIKey() error = %v, wantErr
        %v", err, tt.wantErr)
            }
        })
```

```
    }
}
```

2. TLS/HTTPS Configuration Validation

```
// pkg/security/tls_test.go
package security

import (
    "crypto/tls"
    "testing"
    "time"
)

func TestTLSConfiguration_Security(t *testing.T) {
    // Test TLS configuration security
    config := &tls.Config{
        MinVersion:               tls.VersionTLS12,
        CurvePreferences:         []tls.CurveID{tls.X25519,
        tls.CurveP256},
        PreferServerCipherSuites: true,
        CipherSuites: []uint16{
            tls.TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384,
            tls.TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384,
            tls.TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305,
            tls.TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305,
        },
    }

    // Validate security settings
    if config.MinVersion < tls.VersionTLS12 {
        t.Error("TLS minimum version should be TLS 1.2")
    }

    // Check for secure cipher suites
    insecureCiphers := []uint16{
        tls.TLS_RSA_WITH_AES_128_CBC_SHA,
        tls.TLS_RSA_WITH_AES_256_CBC_SHA,
        tls.TLS_RSA_WITH_3DES_EDE_CBC_SHA,
    }

    for _, cipher := range insecureCiphers {
        for _, configCipher := range config.CipherSuites {
            if cipher == configCipher {
                t.Errorf("Insecure cipher suite detected: %x",
        cipher)
            }
        }
    }
}

func TestCertificate_Validation(t *testing.T) {
    // Test certificate validation
```

```
    cert, err := tls.LoadX509KeyPair("certs/server.crt", "certs/
        server.key")
    if err != nil {
        t.Fatalf("Failed to load certificates: %v", err)
    }

    // Validate certificate
    if time.Now().After(cert.Leaf.NotAfter) {
        t.Error("Certificate has expired")
    }

    if cert.Leaf.Subject.CommonName == "" {
        t.Error("Certificate missing Common Name")
    }
}
```

🎯 DAY 3: INTEGRATION TESTING

End-to-End System Validation

1. Complete Translation Workflow Testing

```
// test/integration/complete_workflow_test.go
package integration

import (
    "context"
    "path/filepath"
    "testing"
    "time"
)

func TestCompleteTranslationWorkflow_AllFormats(t *testing.T) {
    if testing.Short() {
        t.Skip("Skipping complete workflow test in short mode")
    }

    formats := []struct {
        name       string
        inputFile  string
        outputFile string
        provider   string
    }{
        {"FB2 Translation", "testdata/russian_book.fb2",
        "output_sr.fb2", "openai"},
        {"EPUB Translation", "testdata/english_book.epub",
        "output_sr.epub", "zhipu"},
        {"PDF Translation", "testdata/technical_doc.pdf",
        "output_sr.pdf", "deepseek"},
```

```
        {"DOCX Translation", "testdata/business_doc.docx",
        "output_sr.docx", "anthropic"},
        {"HTML Translation", "testdata/web_content.html",
        "output_sr.html", "ollama"},
        {"TXT Translation", "testdata/plain_text.txt",
        "output_sr.txt", "openai"},
    }

    for _, test := range formats {
        t.Run(test.name, func(t *testing.T) {
            ctx, cancel :=
        context.WithTimeout(context.Background(), 10*time.Minute)
            defer cancel()

            // Setup translation system
            translator := NewUniversalTranslator()
            translator.SetProvider(test.provider)

            // Perform translation
            err := translator.TranslateFile(
                ctx,
                test.inputFile,
                test.outputFile,
                "auto", // auto-detect source
                "sr",   // Serbian target
            )

            if err != nil {
                t.Fatalf("Translation failed: %v", err)
            }

            // Validate output
            validateTranslationOutput(t, test.outputFile,
        test.provider)

            // Check quality metrics
            quality, err :=
        translator.GetQualityScore(test.outputFile)
            if err != nil {
                t.Fatalf("Quality assessment failed: %v", err)
            }

            if quality < 0.7 {
                t.Errorf("Translation quality too low: %f",
        quality)
            }
        })
    }
}

func validateTranslationOutput(t *testing.T, outputFile, provider
        string) {
    // File exists and is not empty
    info, err := os.Stat(outputFile)
```

```
    if os.IsNotExist(err) {
        t.Error("Output file was not created")
        return
    }

    if info.Size() == 0 {
        t.Error("Output file is empty")
        return
    }

    // Format-specific validation
    ext := filepath.Ext(outputFile)
    switch ext {
    case ".fb2":
        validateFB2Structure(t, outputFile)
    case ".epub":
        validateEPUBStructure(t, outputFile)
    case ".pdf":
        validatePDFStructure(t, outputFile)
    case ".docx":
        validateDOCXStructure(t, outputFile)
    case ".html":
        validateHTMLStructure(t, outputFile)
    case ".txt":
        validateTXTContent(t, outputFile)
    }
}
```

2. Distributed System Integration Testing

```
// test/integration/distributed_system_test.go
package integration

import (
    "context"
    "testing"
    "time"
)

func TestDistributedSystem_FullWorkflow(t *testing.T) {
    if testing.Short() {
        t.Skip("Skipping distributed test in short mode")
    }

    // Setup mock cluster
    cluster := setupMockCluster(t, 5)
    defer cluster.Shutdown()

    // Prepare batch translation job
    job := &BatchTranslationJob{
        Files:        []string{"testdata/book1.fb2", "testdata/
        book2.epub", "testdata/book3.pdf"},
```

```
            TargetLang:   "sr",
            Provider:     "openai",
            QualityThreshold: 0.8,
        }

        // Execute distributed translation
        ctx, cancel := context.WithTimeout(context.Background(),
            15*time.Minute)
        defer cancel()

        results, err := cluster.ProcessBatch(ctx, job)
        if err != nil {
            t.Fatalf("Distributed batch processing failed: %v", err)
        }

        // Validate results
        if len(results) != len(job.Files) {
            t.Errorf("Expected %d results, got %d", len(job.Files),
            len(results))
        }

        for i, result := range results {
            if result.Error != nil {
                t.Errorf("Translation failed for file %s: %v",
            job.Files[i], result.Error)
                continue
            }

            if result.QualityScore < job.QualityThreshold {
                t.Errorf("Quality below threshold for file %s: %f",
            job.Files[i], result.QualityScore)
            }
        }

        // Validate load balancing
        workerLoads := cluster.GetWorkerLoads()
        if !isLoadBalanced(workerLoads) {
            t.Errorf("Load not balanced across workers: %v",
            workerLoads)
        }
    }

    func TestDistributedSystem_FailoverRecovery(t *testing.T) {
        if testing.Short() {
            t.Skip("Skipping failover test in short mode")
        }

        cluster := setupMockCluster(t, 3)
        defer cluster.Shutdown()

        // Simulate worker failure
        workerID := cluster.GetWorkerIDs()[0]
        err := cluster.SimulateWorkerFailure(workerID)
```

```
    if err != nil {
        t.Fatalf("Failed to simulate worker failure: %v", err)
    }

    // Start new job to test failover
    job := &BatchTranslationJob{
        Files:      []string{"testdata/test.fb2"},
        TargetLang: "sr",
        Provider:   "openai",
    }

    ctx, cancel := context.WithTimeout(context.Background(),
        5*time.Minute)
    defer cancel()

    results, err := cluster.ProcessBatch(ctx, job)
    if err != nil {
        t.Fatalf("Failover failed: %v", err)
    }

    // Verify job completed despite worker failure
    if len(results) != 1 || results[0].Error != nil {
        t.Error("Job failed to complete after worker failure")
    }
}
```

3. API Integration Testing

```
// test/integration/api_integration_test.go
package integration

import (
    "bytes"
    "encoding/json"
    "net/http"
    "testing"
    "time"
)

func TestAPI_CompleteWorkflow(t *testing.T) {
    // Start test server
    server := startTestServer(t)
    defer server.Close()

    client := &http.Client{Timeout: 5 * time.Minute}

    // Test file upload
    uploadResp := uploadTestFile(t, client, server.URL, "testdata/
        test.fb2")
    if uploadResp.StatusCode != http.StatusOK {
        t.Errorf("Upload failed with status: %d",
        uploadResp.StatusCode)
    }
```

```
var uploadResult map[string]interface{}
json.NewDecoder(uploadResp.Body).Decode(&uploadResult)
fileID := uploadResult["file_id"].(string)

// Start translation
translateReq := map[string]interface{}{
    "file_id":      fileID,
    "target_lang":  "sr",
    "provider":     "openai",
    "quality":      0.8,
}

body, _ := json.Marshal(translateReq)
translateResp, err := client.Post(
    server.URL+"/api/v1/translate",
    "application/json",
    bytes.NewBuffer(body),
)
if err != nil {
    t.Fatalf("Translation request failed: %v", err)
}
defer translateResp.Body.Close()

if translateResp.StatusCode != http.StatusOK {
    t.Errorf("Translation request failed with status: %d",
    translateResp.StatusCode)
}

var translateResult map[string]interface{}
json.NewDecoder(translateResp.Body).Decode(&translateResult)
jobID := translateResult["job_id"].(string)

// Monitor progress
jobStatus := monitorJobProgress(t, client, server.URL, jobID)
if jobStatus.Status != "completed" {
    t.Errorf("Translation job failed with status: %s",
    jobStatus.Status)
}

// Download result
downloadResp, err := client.Get(server.URL + "/api/v1/
    download/" + jobID)
if err != nil {
    t.Fatalf("Download request failed: %v", err)
}
defer downloadResp.Body.Close()

if downloadResp.StatusCode != http.StatusOK {
    t.Errorf("Download failed with status: %d",
    downloadResp.StatusCode)
}
```

```
    // Validate downloaded file
    validateDownloadedFile(t, downloadResp.Body, "sr")
}
```

🎯 DAY 4: USER ACCEPTANCE TESTING

Real-World Scenario Testing

1. Production Environment Simulation

```
#!/bin/bash
# production_simulation.sh - Simulate production environment

echo "Setting up production simulation environment..."

# Setup realistic data volumes
mkdir -p production_test/{input,output,logs,config}

# Generate realistic test data (1000+ files)
python3 << 'EOF'
import os
import random
import string
from pathlib import Path

def generate_test_ebook(count=1000):
    """Generate realistic test ebook files"""
    languages = ['ru', 'en', 'de', 'fr']
    formats = ['fb2', 'epub', 'pdf', 'docx']

    for i in range(count):
        lang = random.choice(languages)
        fmt = random.choice(formats)
        size = random.randint(1000, 100000)  # words

        content = f"Test ebook {i} in {lang}, {fmt} format, {size}
        words.\n"
        content += " ".join(random.choices(string.ascii_letters +
        " ", k=size*5))

        filename = f"test_{i:04d}_{lang}.{fmt}"
        filepath = Path("production_test/input") / filename

        with open(filepath, 'w') as f:
            f.write(content)

    print(f"Generated {count} test ebook files")

generate_test_ebook(1000)
EOF
```

```
# Setup production-like configuration
cat > production_test/config/production.json << 'EOF'
{
  "translation": {
    "provider": "openai",
    "model": "gpt-4",
    "max_concurrent": 10,
    "batch_size": 1000,
    "timeout": "30m"
  },
  "storage": {
    "backend": "postgres",
    "connection": "postgresql://user:pass@localhost/
        translator_prod",
    "cache": {
      "backend": "redis",
      "connection": "redis://localhost:6379"
    }
  },
  "logging": {
    "level": "info",
    "file": "production_test/logs/translator.log",
    "rotation": "daily"
  },
  "performance": {
    "memory_limit": "8GB",
    "cpu_limit": 4,
    "enable_caching": true
  }
}
EOF

echo "Production simulation environment ready!"
echo "Input files: $(find production_test/input -name '*' | wc
        -l)"
echo "Configuration: production_test/config/production.json"
```

2. Load Testing

```
// test/performance/load_test.go
package performance

import (
    "context"
    "fmt"
    "runtime"
    "sync"
    "testing"
    "time"
)

func TestLoad_HighConcurrency(t *testing.T) {
```

```
if testing.Short() {
    t.Skip("Skipping load test in short mode")
}

const numGoroutines = 100
const requestsPerGoroutine = 50

var wg sync.WaitGroup
var successCount int64
var errorCount int64
var totalLatency time.Duration

start := time.Now()

for i := 0; i < numGoroutines; i++ {
    wg.Add(1)
    go func(id int) {
        defer wg.Done()

        for j := 0; j < requestsPerGoroutine; j++ {
            reqStart := time.Now()

            err := simulateTranslationRequest(id, j)
            latency := time.Since(reqStart)

            if err != nil {
                atomic.AddInt64(&errorCount, 1)
                t.Logf("Request failed (goroutine %d, request
    %d): %v", id, j, err)
            } else {
                atomic.AddInt64(&successCount, 1)
                atomic.AddInt64((*int64)(&totalLatency),
    int64(latency))
            }
        }
    }(i)
}

wg.Wait()
duration := time.Since(start)

totalRequests := numGoroutines * requestsPerGoroutine
successRate := float64(successCount) / float64(totalRequests)
    * 100
avgLatency := totalLatency / time.Duration(successCount)
throughput := float64(totalRequests) / duration.Seconds()

t.Logf("Load Test Results:")
t.Logf("  Total Requests: %d", totalRequests)
t.Logf("  Success Rate: %.2f%%", successRate)
t.Logf("  Error Rate: %.2f%%", 100-successRate)
t.Logf("  Average Latency: %v", avgLatency)
t.Logf("  Throughput: %.2f req/sec", throughput)
```

```
    t.Logf("  Duration: %v", duration)

    // Validate performance criteria
    if successRate < 95 {
        t.Errorf("Success rate too low: %.2f%% (required >95%%)",
        successRate)
    }

    if avgLatency > 5*time.Second {
        t.Errorf("Average latency too high: %v (required <5s)",
        avgLatency)
    }

    if throughput < 100 {
        t.Errorf("Throughput too low: %.2f req/sec (required
        >100)", throughput)
    }

    // Check for memory leaks
    var m runtime.MemStats
    runtime.ReadMemStats(&m)
    if m.Alloc > 500*1024*1024 { // 500MB threshold
        t.Errorf("Memory usage too high: %d MB", m.Alloc/1024/
        1024)
    }
}

func simulateTranslationRequest(goroutineID, requestID int) error
    {
    // Simulate real translation request
    ctx, cancel := context.WithTimeout(context.Background(),
        30*time.Second)
    defer cancel()

    translator := NewUniversalTranslator()
    translator.SetProvider("openai")

    // Simulate different file types and sizes
    testFiles := []string{
        "testdata/small.fb2",
        "testdata/medium.epub",
        "testdata/large.pdf",
        "testdata/test.docx",
    }

    inputFile := testFiles[requestID%len(testFiles)]
    outputFile := fmt.Sprintf("temp_output_%d_%d.epub",
        goroutineID, requestID)

    err := translator.TranslateFile(ctx, inputFile, outputFile,
        "auto", "sr")
    if err != nil {
        return err
    }
```

```
    // Cleanup
    os.Remove(outputFile)
    return nil
}
```

3. Stress Testing

```
// test/performance/stress_test.go
package performance

import (
    "context"
    "testing"
    "time"
)

func TestStress_MaximumLoad(t *testing.T) {
    if testing.Short() {
        t.Skip("Skipping stress test in short mode")
    }

    // Push system to maximum capacity
    maxConcurrent := runtime.NumCPU() * 4
    var wg sync.WaitGroup
    errors := make(chan error, maxConcurrent)

    // Monitor system resources
    monitor := startResourceMonitor(t)
    defer monitor.Stop()

    start := time.Now()

    for i := 0; i < maxConcurrent; i++ {
        wg.Add(1)
        go func(id int) {
            defer wg.Done()

            ctx, cancel :=
        context.WithTimeout(context.Background(), 10*time.Minute)
            defer cancel()

            // Translate large files to stress system
            err := translateLargeEbook(ctx, id)
            if err != nil {
                errors <- fmt.Errorf("Worker %d failed: %v", id,
        err)
            }
        }(i)
    }

    // Wait for completion or timeout
    done := make(chan struct{})
```

```
    go func() {
        wg.Wait()
        close(done)
    }()

    select {
    case <-done:
        duration := time.Since(start)
        t.Logf("Stress test completed in %v", duration)
    case <-time.After(15 * time.Minute):
        t.Error("Stress test timed out")
    }

    // Check for errors
    close(errors)
    errorCount := 0
    for err := range errors {
        t.Error(err)
        errorCount++
    }

    if errorCount > maxConcurrent/10 { // Allow 10% failure rate
        t.Errorf("Too many errors in stress test: %d/%d",
        errorCount, maxConcurrent)
    }

    // Validate system stability
    if monitor.PeakMemoryUsage() > 2*1024*1024*1024 { // 2GB
        t.Errorf("Memory usage too high: %d MB",
        monitor.PeakMemoryUsage()/1024/1024)
    }

    if monitor.PeakCPUUsage() > 95 {
        t.Errorf("CPU usage too high: %.1f%%",
        monitor.PeakCPUUsage())
    }
}

func translateLargeEbook(ctx context.Context, workerID int) error
    {
    // Translate very large file to stress system
    translator := NewUniversalTranslator()
    translator.SetProvider("openai")

    inputFile := fmt.Sprintf("testdata/large_book_%d.fb2",
        workerID%5)
    outputFile := fmt.Sprintf("stress_output_%d.epub", workerID)

    err := translator.TranslateFile(ctx, inputFile, outputFile,
        "ru", "sr")
    if err != nil {
        return err
    }
```

```
    // Cleanup
    os.Remove(outputFile)
    return nil
}
```

🎯 DAY 5: FINAL VALIDATION & LAUNCH PREPARATION

Comprehensive System Validation

1. Health Check Automation

```
#!/bin/bash
# comprehensive_health_check.sh - Complete system validation

echo "Starting comprehensive health check..."

# Test all major components
echo "1. Testing core translation functionality..."
go test ./pkg/translator/... -v -race -timeout=10m

echo "2. Testing distributed system..."
go test ./pkg/distributed/... -v -race -timeout=15m

echo "3. Testing API endpoints..."
go test ./pkg/api/... -v -race -timeout=10m

echo "4. Testing file format parsers..."
go test ./pkg/ebook/... -v -race -timeout=10m

echo "5. Testing security components..."
go test ./pkg/security/... -v -race -timeout=5m

echo "6. Integration testing..."
go test ./test/integration/... -v -race -timeout=30m

echo "7. Performance benchmarking..."
go test ./test/performance/... -v -race -timeout=20m

echo "8. End-to-end workflow testing..."
go test ./test/e2e/... -v -race -timeout=45m

# Generate comprehensive report
echo "Generating health check report..."
go test ./... -coverprofile=health_check_coverage.out
go tool cover -html=health_check_coverage.out -o
        health_check_coverage.html
```

```
# Run linter
echo "Running code quality checks..."
golangci-lint run ./...

# Security scan
echo "Running security vulnerability scan..."
gosec ./...

# Dependency check
echo "Checking for vulnerable dependencies..."
go list -json -m all | nancy sleuth

echo "Health check complete!"
echo "View detailed results: health_check_coverage.html"
```

2. Production Deployment Validation

```
#!/bin/bash
# production_deployment_validation.sh

echo "Validating production deployment readiness..."

# 1. Build validation
echo "1. Building all components..."
make build-all
if [ $? -ne 0 ]; then
    echo "Build failed!"
    exit 1
fi

# 2. Container validation
echo "2. Building and testing containers..."
docker build -t translator:latest .
docker run --rm translator:latest translator --version

# 3. Configuration validation
echo "3. Validating production configurations..."
for config in config.*.json; do
    echo "Validating $config..."
    ./translator -config "$config" -validate
done

# 4. API validation
echo "4. Testing API deployment..."
./translator-server -config config.production.json &
SERVER_PID=$!

sleep 5
curl -f -k https://localhost:8443/api/v1/health
if [ $? -ne 0 ]; then
    echo "API health check failed!"
    kill $SERVER_PID
```

```
    exit 1
fi

kill $SERVER_PID

# 5. Database validation
echo "5. Testing database connectivity..."
./translator -config config.production.json -test-db

# 6. Security validation
echo "6. Testing security configuration..."
./translator -config config.production.json -security-audit

echo "Production deployment validation complete!"
```

3. Launch Checklist Completion

```
#!/bin/bash
# launch_checklist.sh - Final launch preparation

echo "🚀 LAUNCH CHECKLIST - Universal Ebook Translation System"
echo "=================================================="

# Check test coverage
echo "📊 Test Coverage Analysis..."
COVERAGE=$(go test ./... -coverprofile=launch_coverage.out | grep
        "coverage:" | awk '{print $2}' | sed 's/%//')
echo "Current Coverage: ${COVERAGE}%"
if (( $(echo "$COVERAGE >= 95" | bc -l) )); then
    echo "✅ Test coverage meets requirements (>=95%)"
else
    echo "❌ Test coverage below threshold: ${COVERAGE}%"
fi

# Check documentation
echo "📚 Documentation Status..."
DOC_COUNT=$(find documentation/ -name "*.md" | wc -l)
echo "Documentation files: $DOC_COUNT"
if [ $DOC_COUNT -ge 50 ]; then
    echo "✅ Documentation comprehensive"
else
    echo "❌ Documentation insufficient"
fi

# Check website content
echo "🌐 Website Content Status..."
WEB_FILES=$(find Website/content/ -name "*.md" | wc -l)
echo "Website content files: $WEB_FILES"
if [ $WEB_FILES -ge 10 ]; then
    echo "✅ Website content comprehensive"
else
    echo "❌ Website content insufficient"
fi
```

```
# Check video course
echo "📺 Video Course Status..."
VIDEOS=$(find Website/content/video-course/ -name "*.md" | wc -l)
echo "Video course modules: $VIDEOS"
if [ $VIDEOS -ge 12 ]; then
    echo "✅ Video course complete"
else
    echo "❌ Video course incomplete"
fi

# Check security
echo "🔒 Security Audit..."
SECURITY_ISSUES=$(gosec ./... 2>&1 | grep -c "Issues found")
echo "Security issues: $SECURITY_ISSUES"
if [ $SECURITY_ISSUES -eq 0 ]; then
    echo "✅ Security audit passed"
else
    echo "❌ Security issues found"
fi

# Check performance
echo "⚡ Performance Benchmarks..."
BENCHMARK_RESULT=$(go test ./test/performance/... -bench=.
        -run=^$ | tail -1)
echo "Benchmark: $BENCHMARK_RESULT"

# Check dependencies
echo "📦 Dependency Check..."
VULNS=$(go list -json -m all | nancy sleuth 2>&1 | grep -c "High")
echo "High severity vulnerabilities: $VULNS"
if [ $VULNS -eq 0 ]; then
    echo "✅ No high severity vulnerabilities"
else
    echo "❌ High severity vulnerabilities found"
fi

echo ""
echo "🎯 LAUNCH READINESS SUMMARY"
echo "============================"
echo "📊 Coverage: ${COVERAGE}%"
echo "📚 Docs: $DOC_COUNT files"
echo "🌐 Website: $WEB_FILES files"
echo "📺 Videos: $VIDEOS modules"
echo "🔒 Security: $SECURITY_ISSUES issues"
echo "⚡ Performance: $BENCHMARK_RESULT"
echo "📦 Vulnerabilities: $VULNS high"

# Final decision
if (( $(echo "$COVERAGE >= 95" | bc -l) )) && [ $DOC_COUNT -ge
        50 ] && [ $WEB_FILES -ge 10 ] && [ $VIDEOS -ge 12 ] && [
        $SECURITY_ISSUES -eq 0 ] && [ $VULNS -eq 0 ]; then
    echo ""
```

```
    echo "🎉 SYSTEM READY FOR PRODUCTION LAUNCH!"
    exit 0
else
    echo ""
    echo "❌ SYSTEM NOT READY - Address issues above"
    exit 1
fi
```

📋 DELIVERABLES FOR PHASE 4

System Resolution

- [ ] All disabled modules restored and functional
- [ ] All testing.Short() guards removed
- [ ] Import dependencies resolved
- [ ] Memory leaks and performance issues fixed
- [ ] Security vulnerabilities addressed

Performance Optimization

- [ ] Performance benchmarks established
- [ ] Memory usage optimized (< 100MB typical)
- [ ] CPU efficiency improved
- [ ] Concurrent processing optimized
- [ ] Scalability validated

Security Hardening

- [ ] TLS/HTTPS configuration secured
- [ ] API key security validated
- [ ] SQL injection prevention tested
- [ ] XSS protection implemented

- [ ] Security audit passed

Integration Testing

- [ ] End-to-end workflows validated
- [ ] Distributed system tested
- [ ] API integration verified
- [ ] Load testing completed
- [ ] Stress testing passed

User Acceptance

- [ ] Production environment simulation successful
- [ ] Real-world scenario testing complete
- [ ] Performance criteria met
- [ ] User experience validated
- [ ] Launch checklist completed

Launch Preparation

- [ ] Health check automation implemented
- [ ] Production deployment validation complete
- [ ] Documentation final review
- [ ] Marketing materials prepared
- [ ] Support systems operational

This Phase 4 execution plan ensures the Universal Ebook Translation System is production-ready, secure, performant, and fully validated for successful launch.